PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Software Requirements Specification

for

Restaurant Management System

Version 1.0 approved

Prepared by Abhay Dubey H (PES1UG24CS551)

PES University

06 September 2026

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Table of Contents

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Revision History

Name	Date	Reason For Changes	Version
Abhay Dubey H	06 Sep 2026	Initial draft of SRS	1.0

Introduction

Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for Version 1.0 of the Restaurant Management System (RMS), a web-based application for managing table bookings, food ordering and menu management for a single restaurant outlet. This document covers the complete system: the customer-facing booking and ordering interface, the staff/admin interface for managing bookings, orders and the menu, and the underlying Python/Django and Node.js services that support them.

Intended Audience

This document is intended for the developers implementing the system, the course instructors/evaluators reviewing the project, and the restaurant staff and admin who will be the system's end users. Section 2 gives an overall description of the product, Section 3 describes external interfaces, Section 5 itemizes the functional requirements by system feature, and Section 6 covers non-functional requirements.

Product Scope

The Restaurant Management System is a web application that lets customers check table availability, reserve a table, browse the menu and place a food order, while giving restaurant admin/staff a single dashboard to manage the menu, view and manage bookings, and track orders through to completion. Its goals are to reduce manual/telephonic booking errors, give the restaurant real-time visibility into table occupancy and order status, and provide customers a self-service way to book a table and order food.

References

IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications; Django documentation (docs.djangoproject.com); Node.js documentation (nodejs.org/en/docs); project guidelines provided for this course project.

Overall Description

Product Perspective

The Restaurant Management System is a new, self-contained academic project and is not a follow-on to an existing product. It consists of a Django (Python) backend that exposes a REST API and serves the customer/admin web pages, and a Node.js component that handles real-time order-status updates over WebSocket. The two components share a common relational database.

Product Functions

Table booking: check availability, reserve, and cancel a table for a given date/time and party size.
Menu management: admin can add, edit and remove menu items; customers can browse the live menu.
Food ordering: customers can add items to a cart and place an order; staff can update order status.
Order tracking: customers and staff can see live order status (Received / Preparing / Served).
Authentication: role-based accounts for Customer, Staff and Admin.

User Classes and Characteristics

Customer: member of the public who uses the system to book a table and/or order food; no technical expertise assumed.
Restaurant Staff/Waiter: employees who view bookings and orders and update order status; use a simple dashboard.
Admin/Manager: manages the menu, oversees bookings and orders, and manages staff accounts; the most privileged user class.

Operating Environment
The customer- and staff-facing interface runs in a modern web browser (Chrome, Firefox, Edge) on desktop or mobile. The backend runs on a Linux server with Python 3 and Django serving the REST API, alongside a Node.js service (using Socket.io or an equivalent WebSocket library) for real-time order-status push updates. Data is stored in a relational database (e.g., PostgreSQL, or SQLite for the demo deployment).

Design and Implementation Constraints
The backend must be implemented in Python using the Django framework, with the real-time notification service implemented in Node.js, per the project's technology requirement. The system must use a relational database accessed through Django's ORM. Development and demonstration are constrained to the timeline of the course project, which limits the system to a single restaurant outlet (no multi-tenant/multi-branch support) and to the core booking, ordering and menu features.

2.6 Assumptions and Dependencies
It is assumed that users have a stable internet connection while using the system, that the restaurant operates from a single location, and that a table booking corresponds to a single time slot with a fixed turnaround (e.g., 90 minutes). The system depends on a Node.js WebSocket library (e.g., Socket.io) for real-time updates; the project does not assume integration with an external payment gateway or SMS/email provider in this version.

External Interface Requirements

User Interfaces
The system provides: a Home/Menu page for browsing dishes by category; a Table Booking page showing available slots and a booking form; a Cart/Checkout page for reviewing and placing an order; an Order-Tracking page showing live order status; a Login/Sign-up page; and an Admin Dashboard for managing the menu, bookings and orders. Standard actions such as 'Book Table', 'Add to Cart', 'Place Order' and 'Cancel Booking' appear consistently across the relevant pages, and form-validation errors are shown inline next to the offending field.

Software Interfaces
The frontend communicates with a Django REST Framework API (JSON over HTTP/HTTPS) for all booking, menu, order and authentication operations, and with a Node.js/Socket.io service over WebSocket for real-time order-status push updates. Both components read from and write to a shared relational database (e.g., PostgreSQL).

Communications Interfaces
All browser-to-server communication uses HTTPS with JSON request/response bodies for the REST API, and a persistent WebSocket connection for real-time order-status events pushed by the Node.js service. No other communication protocols (e.g., email/SMS) are required for the core version of the system.

Not applicable — the system requires no dedicated hardware interfaces beyond a standard client device (desktop or mobile) with a web browser and an internet connection.

Analysis Models
Key entities identified for the system's entity-relationship model are: User (Customer/Staff/Admin), Table, Booking, MenuItem, Order and OrderItem, with a Booking optionally linked to an Order. The primary use-case actors are Customer, Staff and Admin, with use cases including Book Table, Cancel Booking, Browse Menu, Place Order, Update Order Status and Manage Menu.

System Features

The functional requirements below are organized by system feature: Table Booking Management, Menu Management, Food Ordering and Order Tracking, and User Authentication and Role Management.

Table Booking Management

5.1.1 Description and Priority

Allows customers to check table availability for a chosen date and time and reserve a table in advance, and allows staff to view and manage all bookings from the admin dashboard. Priority: High.

5.1.2 Stimulus/Response Sequences

Customer selects a preferred date, time and number of guests -> system checks the database for free tables in that slot and displays available options -> customer selects a table and confirms -> system creates a booking record, marks the table as reserved for that slot, and displays a confirmation with a booking reference number. If no table is free, the system suggests the closest available slot.

5.1.3 Functional Requirements

REQ-1: The system shall display real-time table availability for a date, time slot and party size entered by the customer.

REQ-2: The system shall prevent double-booking by locking a table for its reserved time slot once a booking is confirmed.

REQ-3: The system shall allow a customer to cancel a booking up to 30 minutes before the reserved time, after which the table is released back into the available pool.

REQ-4: The system shall allow restaurant staff to view, search and manually create or cancel bookings from the admin dashboard.

Menu Management

5.2.1 Description and Priority

Enables the restaurant admin to add, edit, categorize and remove menu items (name, price, description, category, availability) and enables customers to browse the live menu while placing an order. Priority: High.

5.2.2 Stimulus/Response Sequences

Admin logs in to the admin dashboard -> selects Manage Menu -> adds a new item or edits/deletes an existing one, setting its name, price, category and availability -> system saves the change and the update is immediately reflected on the customer-facing menu.

5.2.3 Functional Requirements

REQ-5: The system shall allow an authenticated admin to create, update and delete menu items, including name, description, price, category and availability status.

REQ-6: The system shall display only menu items marked 'available' to customers browsing the menu or placing an order.

REQ-7: The system shall organize menu items into categories (e.g., Starters, Main Course, Desserts, Beverages) for easier browsing.

Food Ordering and Order Tracking

5.3.1 Description and Priority

Allows a customer to add available menu items to a cart and place an order linked to a table booking or a takeaway request, and allows both the customer and restaurant staff to track the order's status as it moves from Received to Preparing to Served. Priority: High.

5.3.2 Stimulus/Response Sequences

Customer browses the menu, adds items to the cart, and places an order -> system creates an order record with status 'Received' and notifies the kitchen/staff view -> staff updates the status to 'Preparing' and then 'Served' as the order progresses -> the customer's order-tracking screen updates in real time to reflect the current status.

5.3.3 Functional Requirements

REQ-8: The system shall allow a customer to add one or more available menu items to a cart and submit them as a single order.

REQ-9: The system shall assign each order a status of Received, Preparing, or Served, and allow staff to update this status.

REQ-10: The system shall push order-status updates to the customer's screen in real time using a WebSocket connection served by the Node.js component.

REQ-11: The system shall reject an order containing a menu item that has since been marked unavailable, and shall notify the customer of which item was removed.

User Authentication and Role Management

5.4.1 Description and Priority

Provides secure sign-up and login for customers, and role-based access for restaurant staff and admin, so that each user class can only access the functions appropriate to their role. Priority: Medium.

5.4.2 Stimulus/Response Sequences

A new user registers with name, email/phone and password -> system creates an account with the role 'Customer' by default -> user logs in -> system authenticates the credentials and issues a session/token that determines which pages and actions (customer, staff, admin) the user can access.

5.4.3 Functional Requirements

REQ-12: The system shall allow customers to register and log in using an email/phone number and password, storing passwords using a secure hashing algorithm.

REQ-13: The system shall restrict menu-management and booking-override functions to users with the Admin or Staff role.

REQ-14: The system shall log a user out and invalidate their session/token after a period of inactivity.

Other Nonfunctional Requirements

Performance Requirements

Page loads and REST API responses should complete within 2 seconds under normal load. The system should support at least 50 concurrent users, consistent with a course-project demo deployment. Order-status updates pushed over WebSocket should reach the customer's screen within 1 second of the status change on the server.

Safety Requirements

As a software-only system, there are no physical safety hazards. To avoid data-related harm, the system must prevent a table from being booked twice for an overlapping time slot, and must not allow an order to be placed for a menu item that is currently marked unavailable.

Security Requirements

User passwords must be stored using a secure hashing algorithm (e.g., Django's built-in PBKDF2/Argon2 hasher), never in plain text. Access to menu-management and admin/staff functions must be restricted by role-based authentication (e.g., session or JWT-based). The system must rely on Django's built-in protections against SQL injection and cross-site scripting, and all traffic must be served over HTTPS.

Software Quality Attributes

The system should be usable by customers with no training (booking and ordering flows should be self-explanatory), maintainable (Django's app structure should separate booking, menu, ordering and authentication into distinct modules), and reliable (booking and order data must not be lost once confirmed). The web interface should be portable across the latest versions of major browsers.

Business Rules

Only users with the Admin role may add, edit or remove menu items. A table cannot be reserved for two overlapping time slots. Orders may only include menu items currently marked available. Staff may update an order's status but may not delete an order or a booking once confirmed. A customer may cancel a booking only up to 30 minutes before the reserved time.

Other Requirements

The system requires a relational database with tables for Users, Tables, Bookings, MenuItems, Orders and OrderItems, as described in Appendix B. No internationalization is required for this version, as the system will be deployed and demonstrated in English only. The project reuses Django's built-in admin interface where appropriate to speed up development of the admin-side CRUD screens for the menu and bookings.

Appendix A: Glossary

RMS: Restaurant Management System, the product described in this SRS.
Admin: A user with full privileges to manage the menu, bookings, orders and staff accounts.
Staff: A restaurant employee (e.g., waiter) who views and updates bookings and orders.
Customer: An end user who books a table and/or places a food order.
Booking: A reservation of a table for a specific date, time and party size.
Order: A set of menu items requested by a customer, tracked through Received, Preparing and Served states.

REST API: An HTTP-based interface, provided by Django REST Framework, through which the frontend and backend exchange JSON data.
WebSocket: A persistent, bidirectional connection, served by the Node.js component, used to push real-time order-status updates.
JWT: JSON Web Token, an optional mechanism used to authenticate API requests.

Appendix B: Field Layouts

An Excel sheet containing field layouts and properties/attributes and report requirements.

Sample sheet with information required to book a table

Field	Length	Data Type	Description	Is Mandatory
Booking ID	10	Numeric	Unique identifier generated for each booking	Y
Table Number	3	Numeric	Identifier of the table being reserved	Y
Customer Name	60	String	Name of the customer making the booking	Y
Customer Phone	15	Alphanumeric	Contact number of the customer	Y
Booking Date	8	Date	Date for which the table is reserved	Y
Booking Time	5	Time	Time slot for which the table is reserved	Y
Number of Guests	2	Numeric	Party size for the booking	Y
Booking Status	20	String	Current status of the booking (Confirmed/Cancelled)	Y

Sample Report Requirements: fields to be included in each report

Booking Report	**Order Report**
Booking ID	Order ID
Table Number	Booking ID
Customer Name	Customer Name
Customer Phone	Order Items
Booking Date	Item Quantity
Booking Time	Item Price
Number of Guests	Total Amount
Booking Status	Order Status
Created At	Placed At
Cancelled At	Served At
	Payment Mode

Appendix C: Requirement Traceability Matrix

Sl. No	Requirement ID	Brief Description of Requirement	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID
1	REQ-1	Real-time table availability display	Booking Module (Django)	Booking ER Diagram	booking/views.py	TC-01	STC-01
2	REQ-2	Prevent double-booking of a table	Booking Module (Django)	Booking Sequence Diagram	booking/models.py	TC-02	STC-01
3	REQ-3	Allow booking cancellation	Booking Module (Django)	Booking Sequence Diagram	booking/views.py	TC-03	STC-01

4	REQ-4	Staff view/manage bookings	Booking Module (Django)	Admin Dashboard Design	booking/admin_views.py	TC-04	STC-01
5	REQ-5	Admin CRUD on menu items	Menu Module (Django)	Menu ER Diagram	menu/views.py	TC-05	STC-02
6	REQ-6	Show only available menu items	Menu Module (Django)	Menu ER Diagram	menu/serializers.py	TC-06	STC-02
7	REQ-7	Categorize menu items	Menu Module (Django)	Menu ER Diagram	menu/models.py	TC-07	STC-02
8	REQ-8	Cart and order placement	Ordering Module (Django)	Order Sequence Diagram	orders/views.py	TC-08	STC-03
9	REQ-9	Order status transitions	Ordering Module (Django)	Order State Diagram	orders/models.py	TC-09	STC-03
10	REQ-10	Real-time order-status push	Node.js Service	Order State Diagram	realtime/server.js	TC-10	STC-03
11	REQ-11	Reject unavailable items in order	Ordering Module (Django)	Order Sequence Diagram	orders/views.py	TC-11	STC-03
12	REQ-12	Customer registration/login	Auth Module (Django)	Auth Sequence Diagram	accounts/views.py	TC-12	STC-04
13	REQ-13	Role-based access control	Auth Module (Django)	Auth Sequence Diagram	accounts/permissions.py	TC-13	STC-04
14	REQ-14	Session/token expiry on inactivity	Auth Module (Django)	Auth Sequence Diagram	accounts/middleware.py	TC-14	STC-04